

# Singleton VS Static Class in C# with Examples

In this article, I will discuss the **Singleton vs. Static Class in C# with Examples**. Before proceeding, I strongly recommend you read the [Singleton Design Pattern](#) and [Static Class in C#](#) articles.

When I took the Interviews, I generally asked a few questions related to static and singleton classes, as we used these two classes very frequently in our projects. I generally asked **why we need to use a Singleton class if a Static class serves the purpose**. Sometimes, I asked the same question differently: **What is the difference Between Singleton and Static Class in C#, and when to use one over another in project development?** So, at the end of this article, you will get a very good understanding of static and singleton classes in C#. As part of this article, we will discuss the following important pointers.

1. **What are the Similarities Between Singleton and Static class in C#?**
2. **What are the Differences Between Singleton vs Static class in C#?**
3. **Memory Management of Static Class and Singleton Class in C#.**
4. **Real-Time Examples of Static Class and Singleton Class in C#.**

## What are the Similarities Between Singleton and Static Class in C#?

Before discussing the differences between them, let's first discuss the similarities between Singleton vs Static Class in C#

1. Static Class and Singleton Class can have only one instance available in memory throughout the application.
2. They both hold the global state of an application that will be common for all clients.
3. Both Static Classes and Singleton Classes can be implemented as Thread-Safe.

## What are the Differences Between Singleton and Static Class in C#?

The most important point you need to remember is that Static is a language feature, whereas Singleton is a Design Pattern. So, both belong to two different areas. With this in mind, let's proceed and discuss the differences between Singleton vs Static class in C#.

1. We cannot create an instance of a static class in C#. Yes, one copy of the static class is available in memory, but we cannot create an instance of the static class as a developer. But, as a developer, we can create a single instance of a singleton class, and then we can reuse that singleton instance at many different places in the application.
2. When the compiler compiles the static class internally, it treats it as an Abstract and Sealed Class in C#. This is why we neither create an instance nor use the static class as the child class in inheritance. On the other hand, we can create a single instance of the Singleton class, as we can also use the Singleton class as a child class in C#.
3. The Singleton Class Constructor is always marked as private. This is why we cannot create an instance outside the singleton class. It provides either a public static property or a public static method whose job is to create the singleton instance only once and then return that singleton instance each and every time we call that public static property/method from outside of the singleton class.
4. A Singleton class can be initialized lazily or loaded automatically by CLR (Common Language Runtime) when the program or namespace containing the Singleton class is loaded, i.e., Eager Loading. A static class is generally initialized when it is loaded for the first time, and this may lead to potential classloader issues.
5. It is impossible to pass the static class as a method parameter in C #, whereas we can pass the singleton instance as a method parameter in C#.
6. In C#, it allows inheritance with the Singleton class. The Singleton class can be created as a Child class only. You cannot create child classes from the Singleton class. These are not possible with a static class. So, the Singleton class is more flexible than the Static Classes in C#.

## Memory Management of Static Class vs Singleton Class in C#:

When the class is loaded, memory for static classes is allocated once in the high-frequency heap (a special heap for static data). This memory allocation is fixed and exists for the lifetime of the application. Since static classes are not instantiated and their resources are allocated directly in the memory, they live throughout the application life cycle and are only cleaned up when the application domain unloads, or the application exits.

Memory for a singleton instance is allocated on the heap when the instance is created, usually the first time it is requested. This memory remains allocated as long as there is a reference to the instance. The memory for the singleton instance is allocated when the instance is created and normally lasts for the duration of the application. However, unlike static classes, it's possible to implement patterns like using a weak reference for the singleton instance, which allows for the instance to be garbage collected if there are no more references to it.

## The Singleton Class Example in C#

```
public class Singleton
{
    private static Singleton _instance;
    private Singleton() { }
    public static Singleton Instance
    {
        get
        {
            if (_instance == null)
            {
                _instance = new Singleton();
            }
            return _instance;
        }
    }
    public void DoSomething()
    {
        // Method implementation
    }
}
```

## The Static Class Example in C# – Temperature Converter

The following is an example of a static class containing two methods that convert temperature from Celsius to Fahrenheit and Fahrenheit to Celsius. We are sure the formulas for converting temperature from Celsius to Fahrenheit and Fahrenheit to Celsius will not change. Hence, we can use static classes with static methods which do the conversion for us, as shown below.

```
public static class TemperatureConverter
{
    public static double CelsiusToFahrenheit(string temperatureCelsius)
    {
        // Convert argument to double for calculations.
        double celsius = Double.Parse(temperatureCelsius);
        // Convert Celsius to Fahrenheit.
        double fahrenheit = (celsius * 9 / 5) + 32;
        return fahrenheit;
    }
    public static double FahrenheitToCelsius(string temperatureFahrenheit)
    {
        // Convert argument to double for calculations.
        double fahrenheit = Double.Parse(temperatureFahrenheit);
        // Convert Fahrenheit to Celsius.
        double celsius = (fahrenheit - 32) * 5 / 9;
        return celsius;
    }
}
```

```

class Program
{
    static void Main()
    {
        Console.WriteLine("Please select the convertor direction");
        Console.WriteLine("1. From Celsius to Fahrenheit.");
        Console.WriteLine("2. From Fahrenheit to Celsius.");
        Console.Write(":");
        string selection = Console.ReadLine();
        double F, C = 0;
        switch (selection)
        {
            case "1":
                Console.Write("Please enter the Celsius temperature: ");
                F =
TemperatureConverter.CelsiusToFahrenheit(Console.ReadLine());
                Console.WriteLine("Temperature in Fahrenheit: {0:F2}", F);
                break;
            case "2":
                Console.Write("Please enter the Fahrenheit temperature: ");
                C =
TemperatureConverter.FahrenheitToCelsius(Console.ReadLine());
                Console.WriteLine("Temperature in Celsius: {0:F2}", C);
                break;
            default:
                Console.WriteLine("Please select a convertor.");
                break;
        }
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}

```

Run the application and see the output:

```

Please select the convertor direction
1. From Celsius to Fahrenheit.
2. From Fahrenheit to Celsius.
:1
Please enter the Celsius temperature: 20
Temperature in Fahrenheit: 68.00
Press any key to exit.

```

## Singleton vs Static Class in C#: When to Use Which?

Use Singleton when:

- You might need to switch from a single instance to multiple instances in the future without changing the API.
- You require lazy initialization or initialization upon first access.
- The class needs to maintain its state for its lifetime.
- You want the class to implement interfaces or manage inheritance hierarchies.

## Use Static Class when:

- You are creating utility functions that do not store any state.
- You want a simple and straightforward mechanism to provide global access without any initialization logic.
- You need constants, static configuration settings, or methods that are logically grouped together but do not need to interact with instance data.

The choice between using a Singleton or a static class in C# depends on the specific requirements of your application. A Singleton is more appropriate if you need object-oriented features such as inheritance, interface implementation, and state management. A static class is simpler and more efficient if you need a container for stateless functions or static data that is accessed globally.

In the next article, I will discuss one [Real-Time Example of the Singleton Design Pattern in C#](#) using the ASP.NET MVC application. In this article, I try to explain **Singleton VS Static Class in C#** with Examples. I hope this Singleton VS Static Class in C# with Examples article will help you with your needs.